# 高性能异步多模态智能体架构 (xy-core)

执行摘要

**说明**: 本摘要以仓库内 `legacy/mvp_core` 的可复现实验基准与实现为准。

## 一、核心架构：Two-Tier Hybrid（控制平面 vs 数据平面）

针对 Python 在多模态负载（LLM 推理、VL 视觉理解、图像生成等）中易出现的 GIL 争用与事件循环饥饿（event loop starvation）问题，xy-core 采用 **Two-Tier Hybrid Architecture**：将编排与路由（控制平面）与计算执行（数据平面）解耦，使控制路径保持轻量、可预测。

- **Control Plane (L1 Router)**：FastAPI WebSocket 接入 + `GlobalTaskScheduler`。负责优先级/队列化、A/B 路由、故障回退（fallback），并对重型任务（例如图像生成）通过队列容量与并发上限实现背压。
- **Data Plane (L2 Compute)**：承载计算密集型负载。默认使用 Python 线程池卸载（thread offloading），并可选启用 C++ 调度器（`CPPSchedulerEngine/pybind11`）在 GIL 临界路径之外执行与隔离资源。

## 二、实测性能数据（RTX 5070 Laptop GPU 8GB，2025-12-23）

数据来源：
`legacy/mvp_core/experiment_results/xy_core_real.json`
与 `legacy/mvp_core/experiment_results/xy_core_cpp_sched_real.json`。

### 1. 控制平面响应性（Event Loop Lag）

- **xy-core（Python）**：最大漂移 **15.72ms**，平均 **9.55ms**。
- **xy-core（C++ Scheduler）**：最大漂移 **26.28ms**，平均 **9.62ms**。
- **结论**：在真实多模态链路下，控制平面仍保持毫秒级漂移区间，未出现长时间阻塞尖峰。

### 2. 吞吐与端到端时延

- **吞吐（并发 10）**：Python **0.0386 RPS**；C++ Scheduler **0.0403 RPS**。
- **P50/P95（Exp3）**：Python **35.65s / 40.69s**；C++ Scheduler **30.93s / 34.05s**（中位数约 **-13%**，P95 约 **-16%**）。
- **结论**：Two-Tier 将重负载从控制平面剥离后，能在不牺牲交互性的前提下提升链路可预测性与尾延迟表现。

### 3. 可观测性产物 (Observability Artifacts)

- **Trace Timeline**：记录任务排队（scheduled→started）与执行（started→completed）分段，用于定位背压、并行重叠与瓶颈。
- **VRAM Time Series**：通过 NVIDIA-SMI 采样显存占用随时间变化，辅助验证 8GB 显存下的资源隔离策略。

## 三、关键技术点

- **准入控制与背压**：对 GPU 重型任务设置队列容量与并发上限，避免无限堆积导致的 OOM 与尾延迟放大。
- **异步卸载与隔离**：对 CPU-bound/GPU-bound 任务分层调度，确保控制平面不承担计算密集型执行。
- **可复现实验链路**：统一输出 Real/Mock 的结果 JSON、图表与 Trace/VRAM 产物，支撑报告级对比。

## 四、总结

xy-core 通过 Two-Tier Hybrid（Control Plane vs Data Plane）将编排与计算解耦，在 8GB 显存的消费级 GPU 上实现了可预测的交互响应性与可解释的执行轨迹。实测数据显示：在真实多模态链路下，控制平面保持毫秒级漂移，同时端到端中位数与尾延迟显著下降，具备工程落地与持续优化的可行性。